

EFC: Support Concept Easy Automation Lib

Document Index: R01

2015-11-13

Frank Wolz, DC-IA/EDH4

TABLE OF CONTENTS

1	Document Information	3
1.1	Overview	3
1.2	Document History	3
1.3	Referenced Documents	3
2	Introduction	4
3	Basic Considerations on EFC	6
3.1	Preparation	6
3.2	Support of Parametrization Methods	6
3.3	Connection Properties	7
3.4	Supported Fieldbus Process Data	7
3.5	Oscilloscope Parameters	9
4	Supported Methods.....	10
4.1	MoveFrequency	10
4.2	Move Velocity	10
4.3	Move Torque	11
4.4	Limiting Frequency/Velocity/Torque	11
4.5	Switching Drive On/Off	12
4.6	Get/Set Condition	12
4.7	Displaying and Clearing Errors	13
4.8	Drive Reboot	13
4.9	Set/Get IP Suite (IP Addr, Subnet Mask, Gateway) / MAC Address	13
4.10	Set/Get Application Name	14
4.11	Oscilloscope Function	14
4.12	Miscellaneous	14

1 Document Information

1.1 Overview

Project	---
Project Manager	---
Project Number	---
Title	EFC: Support Concept Easy Automation Lib
Version	R01
Created	2015-10-10
Author	Frank Wolz
Department	DC-IA/EDH4
Last saved at	2015-11-13 08:45
File Name	EFC_SupportConceptEAL_R01.docx

Table 1: Overview

1.2 Document History

Document Index	Date (CheckIn)	Created / Changed	Verified	Confirmed	Description
R01	2015-11-13	F. Wolz			First Edition

Table 2: Document History

1.3 Referenced Documents

No.	Document Title and/or Document Type	File Name (Hyperlink included)
001	EDH Document Template	EDH Dokumentvorlage Word ENG V3.1.dot

Table 3: Referenced Documents

2 Introduction

This document specifies a concept for supporting Rexroth Frequency Converter family EFC by Easy Automation Library. EAL standard communication protocol is Sercos/IP, which is supported by EFC's Multi-Ethernet Platform (server). The EFC Multi-Ethernet Platform (MEP) is an option card supporting various Industrial Ethernet derivatives: PROFINET, EtherNet/IP, Sercos III, EtherCAT, Modbus TCP. Sercos/IP protocol is supported by MEP with all Industrial Ethernet derivatives: With PROFINET, EtherNet/IP and Modbus TCP, both, Sercos/IP UDP and TCP services can directly be applied. With Sercos III, communication runs at UCC and with EtherCAT, communication protocol is transmitted via EoE.

Current MEP (01V02) supports the following Sercos/IP services:

Service	UDP support	TCP support
SupportedIpsClassesRequest (49)	✓	
SupportedUdpServicesRequest (61)	✓	
ConnectRequest (63)		✓
PingRequest (65)	✓	✓
ReadEverythingRequest (69)	✓	✓
ReadOnlyDataRequest (71)	✓	✓
ReadDescriptionRequest (73)	✓	✓
WriteNameRequest (75)	✓	✓
WriteAttributeRequest (77)	✓	✓
WriteUnitRequest (79)	✓	✓
WriteMinMaxRequest (81)	✓	✓
WriteDataRequest (83)	✓	✓
WriteDataBits (85)	✓	✓
ReadDataStatusRequest (87)	✓	✓
NameplateRequest (89)	✓	✓
BrowseRequest (91)	✓	
IdentifyRequest (93)	✓	
SetIpRequest (95)	✓	
ResetRequest (97)	✓	
NameplateBroadcastRequest (99)	✓	
WatchdogRequest (101)	✓	✓
InterfacesRequest (103)	✓	✓
ReadSegment (109)	✓	✓
ReadSamplesRequest (111)	✓	✓

Hint: With Sercos/IP services, Function Codes at MEP or EFC can be addressed in two ways: a) transformed into its appropriate P parameter EIDN representation or b) by directly transmitting the numerical representation of a Function Code using the "extended field". When using either a) or b), the "opposite" addressing field (EIDN or extended field, respectively) has to be zero.

Controlling EFC using Sercos III

With selected fieldbus derivate "Sercos III", the MEP supports three control modes:

- **CYCLIC MODE:** Communication via regular Sercos III telegrams (MDT/AT).
- **ACYCLIC MODE:** Without running cyclic Sercos III communication. Process Data Configuration can be set by writing H3.30/H3.31. Then, Sercos subdevice state machine at MEP can be switched to OL by command S-0-0422. Now, S-0-0134/S-0-0135 and process data parameters can be read/written by Sercos/IP services. Note, that from EFC base system's view, process data exchange with MEP looks like a regular fieldbus session. It is recommended to activate the Sercos/IP watchdog in acyclic mode.
- **POLLING MODE:** Without running cyclic Sercos III communication and without using S-0-0134/S-0-0135 and without specifying process data configuration. Drive is controlled only by reading/writing EFC parameters directly.

⇒ EAL should support EFC using acyclic mode with activated watchdog.

3 Basic Considerations on EFC

3.1 Preparation

First, a Sercos/IP connection has to be established to EFC. The IP address is configured via [S-0-1020], [S-0-1021], [S-0-1022] and [S-0-1039]. These settings will become active after executing [S-0-1048]. Default IP address is 192.168.0.1.

Method Prototype:

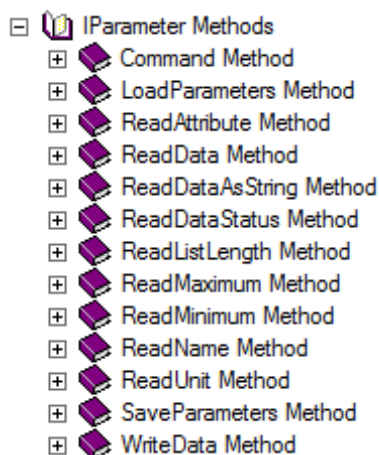
```
void Initialize()
```

Implementation: In order to control EFC using Sercos III acyclic mode, some parametrization must be done before on EFC:

- Communication Protocol [E8.00]: Set to value 1 for "Option Card"
- First frequency setting source [E0.00]: Set to value 20 for "Communication"
- First run command source [E0.01]: Set to value 2 for "Communication"
- Select Sercos III fieldbus by writing string "S3" to [H3.40].
- Set H3.30 to List [H0.01], [d0.02].
- Set H3.31 to List [H0.00], [H0.10].
- Switch MEP into Operational Level (OL) by executing [S-0-0422].
- Optional: Activate Watchdog by running Sercos/IP WatchdogRequest-Service (101)

3.2 Support of Parametrization Methods

Functions:



Implementation: Use appropriate Sercos/IP read/write services.

Parametrization via Function Codes

- ➔ New Interface at EAL for using FCs (Strings) - support both, EIDN and FC.
- ➔ Conversion algorithm for FCs to P-EIDNs needed

Example:

```
// Reads parameter E0.07. Read method is always synchronous.
object settingfreqObj = axis.Parameter.ReadData("E0.07");
```

3.3 Connection Properties

Name	Description	Supported by EFC
Executed Asynchronously	Executes task asynchronously	(YES)
IsConnected	True if this object is connected to the real drive.	(YES)
IsQueueCompleted	True, if all task from the queue are completed.	(YES)
Logic	Property to access all logic related methods and properties	NO
Motion	Property to access all Motion related methods and properties	YES
Oscilloscope	Property to access the Oscilloscope functionality	YES
Parameter	Parameter property, which holds all parameter related methods.	YES
System	Property to access all System related methods and properties	YES

3.4 Supported Fieldbus Process Data

With EFC01, only selected parameters can be configured for process data. Following tables give an overview in terms of EFC firmware 03V08. (Tables are not complete.)

Fieldbus Consumer Data (b8.61):

Function Code	Hexadecimal Representation	Name	Signed Value	Data Type	Decimals
H0.00	0x6000	Control Word (see above)		WORD	0
H0.10	0x600A	Frequency Command [0.01Hz]		WORD	2
H0.11	0x600B	Torque Command [Nm?] – Not implemented, yet – planned.		WORD	?

Fieldbus Producer Data (b8.62):

Function Code	Hexadecimal Representation	Name	Signed Value	Data Type	Decimals
H0.01	0x6001	Status Word (see above)		WORD	0
d0.00	0x1000	Actual output frequency [0.01Hz]		WORD	2
d0.01	0x1001	Actual speed [1 rpm]		WORD	0
d0.02	0x1002	Setting frequency [0.01 Hz]		WORD	2
d0.03	0x1003	Setting speed [1 rpm]		WORD	0
d0.04	0x1004	User-defined setting speed [0.1]		WORD	1
d0.05	0x1005	User-defined output speed [0.1]		WORD	1
d0.10	0x100A	Output voltage [V]		WORD	0
d0.11	0x100B	Output current [0.1 A]		WORD	1
d0.12	0x100C	Output power [0.1 kW]	Y	WORD	1

d0.13	0x100D	DC-bus voltage [V]		WORD	0
d0.16	0x1010	Output Torque [0.1 %]	Y	WORD	1
d0.17	0x1011	Setting torque [0.1 %]	Y	WORD	1
d0.20	0x1014	Power module temperature [°C]	Y	WORD	0
d0.21	0x1015	Actual carrier frequency [1kHz]		WORD	0

Control word **H0.00** and status word **H0.01** are described here:

Drive Control Word (**H0.00**):

Bit	Value	Description
15-8	-	Reserved
7	0	Control word inactive
	1	Active
6	1	Stop acceleration/deceleration active (stop the internal acc/dec ramp generator)
	0	Inactive
5	1	Fault reset active
	0	Inactive
4	1	E-stop active
	0	Inactive
3	1	Stop according to parameter setting
	0	Inactive
2	1	Reverse
	0	Forward
1	1	Jog active (jogging direction determined by bit 2)
	0	Inactive
0	1	Run command active
	0	Inactive

Drive Status Word (**H0.01**):

Bit	Value	Description
15-8	-	Fault code (equal to [E4.20])
7	1	Fault
	0	No fault
6	1	Stall over current
	0	Normal
5	1	Stall over voltage
	0	Normal
4	1	Decelerating
	0	Not in deceleration
3	1	Accelerating
	0	Not in acceleration
2	1	Jogging
	0	Not in jog
1	1	Running
	0	Stop
0	1	Reverse
	0	Forward

3.5 Oscilloscope Parameters

Function Code	Corresponding IndraDrive Parameter	Name
b2.70	P-0-0020	Oscilloscope: operating mode
b2.71	P-0-0021	Oscilloscope: list of measured values 1
b2.72	P-0-0022	Oscilloscope: list of measured values 2
b2.73	P-0-0145	Oscilloscope: list of measured values 3
b2.74	P-0-0146	Oscilloscope: list of measured values 4
b2.75	P-0-0025	Oscilloscope: trigger mask
b2.76	P-0-0035	Oscilloscope: control offset
b2.77	P-0-0027	Oscilloscope: trigger level
b2.78	P-0-0028	Oscilloscope: control word
b2.79	P-0-0029	Oscilloscope: status word
b2.80	P-0-0030	Oscilloscope: trigger edge
b2.81	P-0-0031	Oscilloscope: time resolution
b2.82	P-0-0032	Oscilloscope: size of memory
b2.83	P-0-0033	Oscilloscope: number of measured values after trigger event
b2.84	P-0-0150	Oscilloscope: number of valid measured values
b2.85	P-0-0149	Oscilloscope: Signal configuration list
b2.86	P-0-0036	Oscilloscope: external trigger signal
b2.87	P-0-0037	Oscilloscope: internal trigger signal

For details see subsection 4.11.

4 Supported Methods

This section contains a proposal for supporting EFC by EAL.

4.1 MoveFrequency

Method Prototype:

```
void MoveFrequency(
    double frequency,    // [Hz]
    double acceleration, // [Hz/s]
    double deceleration // [Hz/s]
)
```

Implementation:

1. Read [E0.08] (= Max. Frequency = reference point for accel/decel. settings)
2. Write [E0.26] := [E0.08] / acceleration;
3. Write [E0.27] := [E0.08] / deceleration;
4. Write [H0.10] := | frequency | * 100;
5. If (frequency < 0), then:
 - a. write [H0.00] := 0x85; // Controlword active, RUN, REVERSE
 - else
 - b. write [H0.00] := 0x81; // Controlword active, RUN, FORWARD

Remark: If method is called a second time with changed motor direction, for example, EFC system automatically performs a rampdown / rampup sequence. Hence, this needs not to be considered.

4.2 Move Velocity

Method Prototype:

```
void MoveVelocity(
    double velocity,    // [rpm]
    double acceleration, // [rpm/s]
    double deceleration, // [rpm/s]
    short jerk          // (not supported)
)
```

Units are fixed to rpm, as only rotative motors are supported with EFC.

Implementation:

1. Read [E0.08] (= Max. Frequency = reference point for accel/decel. settings)
2. Read [C1.11] (= Number of Motor Pole Pairs)
3. Write [E0.26] := ([E0.08] * 120) / (acceleration * [C1.11]);
4. Write [E0.27] := ([E0.08] * 120) / (deceleration * [C1.11]);
5. Let frequency = (velocity * [C1.11]) / 120;
6. Set [H0.10] := | frequency | * 100;
7. If (frequency < 0), then:
 - a. Set [H0.00] := 0x85; // Controlword active, RUN, REVERSE
 - else
 - b. Set [H0.00] := 0x81; // Controlword active, RUN, FORWARD

4.3 Move Torque

Method Prototype:

```
void MoveTorque(  
    double torque,      // [Nm]  
    bool   reverse      // [FALSE=FORWARD, TRUE=REVERSE]  
)
```

Implementation:

(Not implemented, yet – planned. Just for info: Torque Mode will be selected via a new bit at H0.00, Torque command value will be set via a new H0.xx parameter.)

4.4 Limiting Frequency/Velocity/Torque

Function Prototypes:

```
double GetFrequencyLimitPos()  
double GetFrequencyLimitNeg()  
double GetVelocityLimitPos()  
double GetVelocityLimitNeg()  
double GetTorqueLimit() - Not implemented, yet - planned
```

Implementation:

```
Read [E0.09], [E0.10] and [C1.11].  
FreqPosLimit = [E0.09] / 100  
FreqNegLimit = [E0.10] / 100  
VelPosLimit = ([E0.09] * 120 / 100) / [C1.11]  
VelNegLimit = ([E0.10] * 120 / 100) / [C1.11]
```

Function Prototypes

```
void SetFrequencyLimitPos(  
    double frequencyLimit  
)  
void SetFrequencyLimitNeg(  
    double frequencyLimit  
)  
void SetVelocityLimitPos(  
    double velocityLimit  
)  
void SetVelocityLimitNeg(  
    double velocityLimit  
)
```

Implementation:

```
Read [C1.11]
FreqPosLimit: Write [E0.09] := frequencyLimit * 100
FreqNegLimit: Write [E0.10] := frequencyLimit * 100
VelPosLimit: Write [E0.09] := ( velocityLimit * [C1.11] ) / 120 * 100
VelNegLimit: Write [E0.10] := ( velocityLimit * [C1.11] ) / 120 * 100
```

4.5 Switching Drive On/Off

Method Prototype:

```
void Power(
    bool value
)
```

Implementation:

```
Read CtrlWord := [H0.00]
If (value == TRUE)
    Write [H0.00] := CtrlWord | 0x0001
Else
    Write [H0.00] := CtrlWord & 0xFFFE
```

Hint: This function also could be implemented by controlling S-0-0134 Bit 14 (Drive Enable).

4.6 Get/Set Condition

Function Prototype:

```
AxisCondition GetCondition()
```

Implementation:

```
If ( [H0.01] & 0x02)
    AxisCondition := AXIS_CONDITION_ACTIVE
else
    AxisCondition := AXIS_CONDITION_DEACTIVATED
```

Function Prototype:

```
void SetCondition(
    AxisCondition condition
)
```

Implementation:

```
Read CtrlWord := [H0.00]
If (condition == AXIS_CONDITION_ACTIVE)
  Write [H0.00] := CtrlWord | 0x0001
Else If (condition == AXIS_CONDITION_DEACTIVATED)
  Write [H0.00] := CtrlWord & 0xFFFE
```

4.7 Displaying and Clearing Errors

Method Prototype:

```
void ClearError()
```

Implementation:

Execute command parameter: S-0-0099

Displaying errors: S-0-0390 and S-0-0095 are supported.

4.8 Drive Reboot

Method Prototype:

```
void Reboot()
```

Implementation:

Execute command parameter: S-0-1350

4.9 Set/Get IP Suite (IP Addr, Subnet Mask, Gateway) / MAC Address

Method Prototypes:

```
void SetIpSuite(
    String^ address,
    String^ subnet,
    String^ gateway
)
```

```
Array^ GetIPSuite(void
)
```

Hint: This new method should be implemented, because setting IP address, subnet and gateway separately could cause consistency problems. (Note, that writing appropriate parameters leads to new values getting active immediately.)

Implementation:

Use sercos/IP service "SetIpRequest" (95). This service sets all three components and switches them active at MEP.

Method Prototype:

```
String^ GetMacAddress( )
```

Implementation:

Read [H3.00]

4.10 Set/Get Application Name

Method Prototype:

```
String^ GetName( )
```

```
void SetName(  
    String^ name  
)
```

Implementation:

Read/Write [S-0-0142]

4.11 Oscilloscope Function

Could be used either with IndraDrive P parameters (as implemented at EAL) or with Function Codes as depicted in section 3.5.

Remarks:

- IOscilloscope.RaiseTrigger(): For EFC01, use P-0-0036 for manually triggering (instead of P-0-0136).
- Contents of oscilloscope parameter lists still always are EIDNs.

Not supportable (because of unsupported parameters):

- IChannel.Signal Property (because lack of P-0-0023/24 etc.)
- ITrigger.Signal and Type Property (because a lack of P-0-0026/27)

4.12 Miscellaneous

Add some device identification methods:

- System type (IndraDrive .., EFC,...)
- System version

Use S-0-1300 ??